Learning Successor Measures from Manipulation Play

Anonymous authors
Paper under double-blind review

Abstract

Behavior foundation models promise zero-shot control: pretrain on offline data, then reuse the learned representation for downstream tasks without task-specific training. A natural framework to achieve this is Successor-measure methods such as Forward–Backward representations (FB), which have shown impressive results on learning diverse locomotion, but can struggle when success depends on precise interactions. For example, in pick-and-place tasks, FB policies can reach an object but fail to maintain a grasp or carry it to a specified goal. We hypothesize that these failures arise from environment features we describe as control bottlenecks, states where choosing the right action is crucial for success. We show that the low-rank squared approximation used by FB can lose the difference between action values at these states, and that replacing FB's action values with rollout estimates at such states removes a fifth of its failures while the object is carried, with the candidate actions and the rest of the policy unchanged. To better preserve relative action values with limited capacity, we introduce Density-FB, which uses a categorical Bellman objective to learn scores for future states and converts them into probabilities rather than fitting occupancies directly. In controlled bottleneck problems, Density-FB takes better actions despite smaller embeddings than vanilla FB. Across OGBench, MetaWorld, and ManiSkill, Density-FB improves manipulation success over FB by up to 42.5, 74.7, and 52.7 percentage points, respectively.

1 Introduction

Reinforcement Learning (RL) algorithms that estimate the Successor-measure (SM) enable zero-shot control by learning future-state occupancies that can be reused for rewards specified after training (???). However, in practice even small errors in the approximation can lead these methods to give the wrong ranking to actions, which matters specifically for problems where this ranking is important for downstream control. In manipulation, for example, FB can generally reach the correct object but fail to execute a stable grasp, or correctly place the object at the goal position (?). This raises a natural question: Why does FB fail to capture such precise behavior?

We hypothesize that this failure arises because of control bottlenecks: states where the choice of action substantially changes the distribution of future states. For example, passing through a doorway determines which side of an environment can subsequently be explored; making the correct contact during an insertion determines whether the object can subsequently be manipulated; and establishing a stable grasp determines whether an object can be carried at all. Although these decisions last only a few steps, their consequences persist much longer. Reliable control therefore requires accurately learning the differences between successful and unsuccessful actions at these states.

Consider grasping and carrying a cube, where the policy must choose actions that keep the cube secure while moving it toward the goal. In fig. 1, FB already holds the cube, but its predicted future occupancy of hte cube positions remains broadly spread across the workspace, resembling the dataset's cube-position distribution, and the extracted policy eventually drops the cube. We hypothesize that FB's low-rank representation captures the broad distribution of cube positions in the dataset but fails to distinguish the future occupancies produced by actions that carry the cube securely toward the goal from those

produced by actions that cause it to slip. With limited capacity, errors at infrequent bottleneck decisions can contribute little to the average prediction error, even when they prevent the model from identifying the action needed for success.

We introduce Density-FB to better preserve the differences between the future occupancies of actions at control bottlenecks. Our idea is simple: train the low-rank approximation to predict how likely different future states are relative to one another. Instead of using the bilinear output directly as future occupancy, we treat it as a score and pass these scores through a softmax over future states. The resulting occupancy is therefore a nonlinear function of the low-rank features, allowing the same representation size to express action-dependent differences that a direct low-rank fit may lose. We learn these probabilities with a categorical Bellman objective. Because the bilinear output now represents a score rather than an occupancy, FB's usual linear value readout no longer applies; we instead evaluate actions by averaging downstream rewards under their predicted future-state distributions.

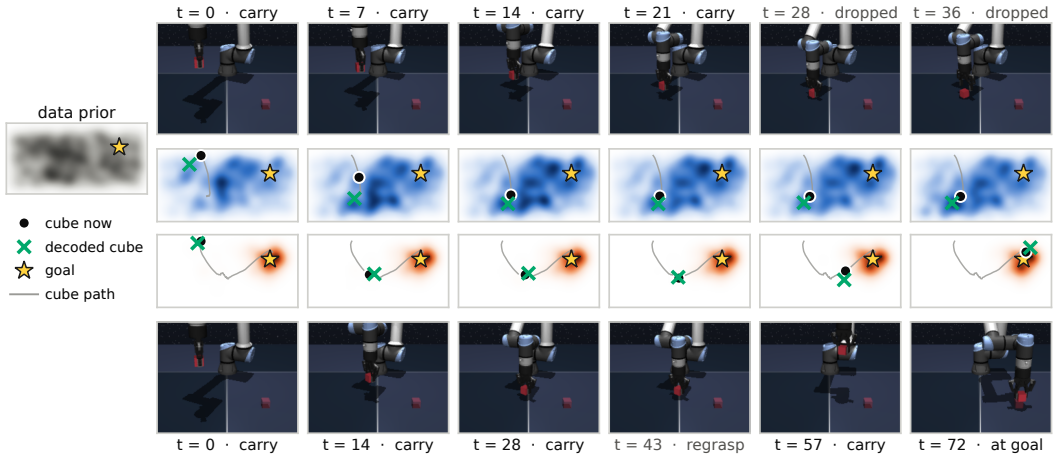


Figure 1: Predicted future occupancies of the cube position and observed behavior. Both methods start from the same state and goal. Blue heatmaps show FB predicting future cube visits broadly across the workspace, resembling the dataset distribution (gray). Orange heatmaps show Density-FB predicting visits concentrated near the goal. Each heatmap summarizes discounted future visits starting from the corresponding state and action under that method's goal-conditioned policy. The top and bottom image rows show the respective rollouts: FB drops the cube, whereas Density-FB reaches the goal after regrasping.

2 Related Work

Successor representations, successor features, and successor measures support transfer across reward functions in MDPs with shared dynamics. We review their use in zero-shot RL, classification-based estimation of discounted state occupancy, categorical value learning, generative models of future states, and methods for mapping downstream rewards to latent task variables.

Successor representations and measures for zero-shot RL. Successor features separate a reward's feature weights from the discounted feature occupancies induced by a policy, enabling transfer across tasks with shared dynamics (?). Forward–Backward representations (FB) factorize successor measures and jointly learn a family of task-conditioned policies, allowing new rewards to be specified after pretraining (??). Proto-Successor Measures learn a basis of successor measures intended to cover the behaviors available in an environment (?). Later work adapts FB to offline datasets with limited coverage (?) and uses autoregressive features and advantage-weighted policy learning to improve offline training (?). We study a different component of these methods: the objective used to estimate the successor measure. Specifically, we replace pointwise squared regression of successor density ratios with categorical estimation of a normalized successor distribution.

Classification-based estimation of state occupancy. C-learning estimates discounted future-state occupancy using recursive binary classification (?). Contrastive RL uses contrastive estimation to learn goal-conditioned value functions (?). TD-InfoNCE instead derives a temporal-difference objective based on categorical classification over future-state samples (?). Our successor-measure estimator uses this categorical construction. We apply it to FB, HILP, and RLDP, and recover action values by integrating arbitrary state-based rewards against the estimated successor distribution. [Aditya: add refs]

Categorical value prediction. Categorical parameterizations are also widely used to learn bootstrapped value targets C51 represents the distribution of returns on a discrete support and trains it using a distributional Bellman update (?). Other methods use categorical encodings as an alternative to regressing scalar value targets. ? show that categorical cross-entropy can improve the performance and scalability of value learning across several domains. Categories for Density-FB are states sampled from the offline distribution, and their probabilities approximate the normalized successor measure. Values for different rewards are computed from this same state distribution after training.

Generative models of discounted future states. Successor distributions can also be represented with generative models. γ-models generate states at geometrically distributed future times using a temporal-difference objective (?). TD-Flow uses flow matching to model long-horizon state distributions for policy evaluation and planning (?). Intention-conditioned flow occupancy models condition these distributions on latent variables inferred from offline trajectories and use them for policy learning (?). In contrast, Density-FB assigns probabilities to future states sampled from the offline state distribution rather than generating new states.

Reward conditioning and task inference. In standard FB, a downstream reward is mapped to a latent task vector through the linear factorization of the successor measure. Soft FB instead searches over pretrained task vectors to optimize a specified utility (?), while optimistic task inference collects additional information during deployment (?). In Density-FB, the bilinear output is a log-density score, so the standard linear mapping from rewards to both values and task vectors no longer holds. We evaluate a reward by integrating it against the predicted successor distribution and map it separately to a task vector in the pretrained policy family. Zero-shot performance therefore depends both on the accuracy of the successor estimate and on whether the pretrained family contains a suitable policy.

3 Preliminaries

Offline reward-free setting. We consider a controlled Markov process $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mu, P)$ and an unlabeled dataset $\mathcal{D} = \{(s_i, a_i, s'_i)\}_{i=1}^N$ collected without downstream reward labels. Here, $\mathcal{S}$ denotes the state space, $\mathcal{A}$ denotes the action space, $\mu \in \Delta(\mathcal{S})$ denotes the initial state distribution, and $P(\cdot \mid s, a) \in \Delta(\mathcal{S})$ denotes the environment transition kernel. Let ρ denote the marginal distribution of next states induced by the offline data-collection process, from which the observed next states $\{s'_i\}_{i=1}^N$ are sampled. Downstream tasks are specified by state rewards $r : \mathcal{S} \to \mathbb{R}$.

Successor measures. For a policy π, define its discounted SM over future states as

$$M^\pi(s, a, X) := (1-\gamma)\mathbb{E}_\pi\left[\sum_{t\geq 0}\gamma^t \mathbf{1}[s_{t+1} \in X] \,\middle|\, s_0 = s,\ a_0 = a\right], \tag{1}$$

for measurable $X \subseteq \mathcal{S}$. It satisfies the Bellman recursion

$$M^\pi(s, a, \cdot) = (1-\gamma)P(\cdot \mid s, a) + \gamma \mathbb{E}_{\substack{s'\sim P(\cdot|s,a)\\ a'\sim\pi(\cdot|s')}}\left[M^\pi(s', a', \cdot)\right]. \tag{2}$$

By construction, $M^\pi(s, a, \cdot)$ is a probability measure,

$$M^\pi(s, a, \mathcal{S}) = (1-\gamma)\sum_{t\geq 0}\gamma^t = 1. \tag{3}$$

For any downstream reward r, integrating against the normalized SM gives the correspondingly normalized discounted return,

$$\int_{\mathcal{S}} r(s^+)\, M^\pi(s, a, ds^+) = (1-\gamma) Q_r^\pi(s, a). \tag{4}$$

Equivalently,

$$Q_r^\pi(s, a) = \frac{1}{1-\gamma} \int_{\mathcal{S}} r(s^+)\, M^\pi(s, a, ds^+). \tag{5}$$

Forward-Backward representations. FB (??) learns a family of policies π_z indexed by task vectors $z \in \mathbb{R}^d$, together with a forward representation $F_\theta(s, a, z) \in \mathbb{R}^d$ and a backward representation $B_\theta(s^+) \in \mathbb{R}^d$. It factorizes the density of the SM relative to the dataset state marginal ρ as an inner product between these representations:

$$M^{\pi_z}(s, a, ds^+) \approx F_\theta(s, a, z)^\top B_\theta(s^+)\, \rho(ds^+). \tag{6}$$

For a dataset transition (s, a, s'), an independently sampled state $s^+ \sim \rho$, and $a' \sim \pi_z(\cdot \mid s')$, FB then learns these representations using the Bellman equation through the squared regression objective:

$$\mathcal{L}_{\mathrm{FB}} = \mathbb{E}\Big[\big(F_\theta(s, a, z)^\top B_\theta(s^+) - \gamma \bar{F}(s', a', z)^\top \bar{B}(s^+)\big)^2 - 2(1-\gamma) F_\theta(s, a, z)^\top B_\theta(s')\Big], \tag{7}$$

where $\bar{F}$ and $\bar{B}$ denote target representations. For a downstream state reward r, FB forms the task vector

$$z_r := \mathbb{E}_{s\sim\rho}\left[r(s) B_\theta(s)\right]. \tag{8}$$

Combining Eq. (5) with the factorization in Eq. (6) gives

$$Q_r^{\pi_z}(s, a) \approx \frac{1}{1-\gamma} F_\theta(s, a, z)^\top z_r. \tag{9}$$

4 A failure mode at control bottlenecks

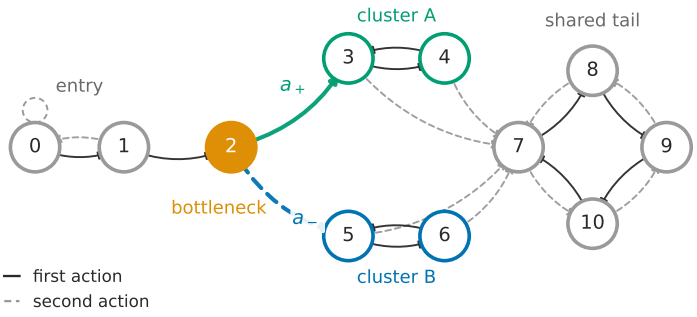


Figure 2: A control bottleneck. At state 2, action a_+ enters cluster A (states $3, 4$), while a_- enters cluster B (states $5, 6$). Both branches lead to the shared tail (states 7–10). Reward is 1 in cluster A and 0 elsewhere, so a_+ has a higher return despite the overlap in the two actions' futures.

Prior work has observed that methods that methods to approximate SMs struggle on robotic manipulation tasks that require precise interactions (???). In particular, ? argue that interactions can change the active dynamics of an object and, consequently, its future reachability. Building on this observation, we call states where the choice of action substantially changes the distribution of future states control bottlenecks.

At such states, a SM approximation must capture how those futures change when the action changes. Under a fixed continuation policy π, each action produces a distribution over future states, and for any downstream reward r, the preference between two actions a_+, a_- at a state s depends on the pairwise action-value difference:

$$Q_r^\pi(s, a_+) - Q_r^\pi(s, a_-) = \frac{1}{1-\gamma} \int r(s^+) \left[M^\pi(s, a_+, ds^+) - M^\pi(s, a_-, ds^+)\right]. \tag{10}$$

Algorithm 1 Density-FB training and zero-shot control

Require: Offline dataset $\mathcal{D}$, successor networks (F_θ, B_θ), target networks $(\bar{F}, \bar{B})$, actor π_ω

1: for each training step do
2: Sample transitions $\{(s_i, a_i, s'_i, \text{terminal}_i)\}_{i=1}^n$ and task vectors $\{z_i\}_{i=1}^n$
3: Compute p_{ij}, $\bar{p}'_{ij}$, and t_{ij} using Equation (15)
4: Update (F_θ, B_θ) using $\mathcal{L}_{\text{DFB}}$
5: Update π_ω using Equation (19), with (F_θ, B_θ) fixed
6: Update target networks $(\bar{F}, \bar{B})$
7: end for

Zero-shot control for downstream reward r

8: Infer z_r from reward-labeled offline states using Appendix B.2
9: Execute the frozen policy $\pi_\omega(\cdot \mid s, z_r)$

FB approximates a linear combinations of shared backward features. To isolate the effect of this representation, we consider least-squares fits of exact SM with the features held fixed. Let m_+, m_- denote the densities of $M^\pi(s, a_+, \cdot)$ and $M^\pi(s, a_-, \cdot)$ relative to the reference distribution ρ. Write $\Delta m = m_+ - m_-$ and $\langle f, g \rangle_\rho = \mathbb{E}_{x\sim\rho}[f(x)g(x)]$.

Lemma 1 (What linear successor approximation retains). Fix backward features $B : \mathcal{S} \to \mathbb{R}^d$. Let Π_B denote least-squares projection onto functions of the form $c^\top B(s^+)$, where $c \in \mathbb{R}^d$. Suppose $m_+, m_- \in L^2(\rho)$, and let $\widehat{m}_+, \widehat{m}_-$ be their least-squares fits using these features. Then, the difference between $m_+(s^+)$ and $m_-(s^+)$ is the true difference restricted to the feature space:

$$\widehat{m}_+(s^+) - \widehat{m}_-(s^+) = (\Pi_B \Delta m)(s^+). \tag{11}$$

For any bounded state reward r, let G_r denote the true action-value difference in (10) and let $\widehat{G}_r = \langle \widehat{m}_+ - \widehat{m}_-, r \rangle_\rho / (1-\gamma)$ denote the fitted action-value difference. Consequently, the error in the estimated advantage is simply the reward of the uncaptured futures:

$$G_r - \widehat{G}_r = \frac{\langle \Delta m - \Pi_B \Delta m, r \rangle_\rho}{1-\gamma}. \tag{12}$$

Figure 2 gives a concrete example. At the bottleneck state 2, action a_+ enters cluster A through state 3, while a_- enters cluster B through state 5. Only states 3 and 4 give reward. Both branches lead to the shared tail of states 7-10, but a_+ visits the rewarded cluster first and therefore has a higher return. Lemma 1 does not say which features FB learns; it says that, whatever they are, the fitted difference between the two futures is their projection of the true one, and the lost action value is the reward-weighted part of the difference that lies outside their span. A feature span that omits the split between the two clusters therefore loses this decision exactly when the reward distinguishes the clusters. Section 6.1 shows that the features FB learns on this process do omit that split, uses the lemma to account for the resulting value error, and compares against our method Density-FB, which we introduce in the next section.

5 Density-FB: Learning Successor Measures as Distributions

Section 4 showed that a linear SM approximation can discard the part of an action-dependent difference outside its feature span. Density-FB uses FB's bilinear output to score future states, then normalizes its exponential to predict their distribution. The resulting density is nonlinear in the backward features. We fit this distribution with a categorical Bellman objective (section 5.1), and since the bilinear output no longer represents a linear map, we then replace FB's linear value readout with an expectation of rewards under the predicted distribution (section 5.2).

5.1 Learning a normalized successor distribution

We parameterize its density relative to ρ as exponential functions:

$$f_\theta(s,a,z,x) := \frac{F_\theta(s,a,z)^\top B_\theta(x)}{\tau}, \qquad h_\theta(s,a,z,x) := \frac{\exp f_\theta(s,a,z,x)}{\mathbb{E}_{u\sim\rho}[\exp f_\theta(s,a,z,u)]}, \tag{13}$$

where $\tau > 0$, and define $\widehat{M}_\theta(s,a,z,dx) = h_\theta(s,a,z,x)\rho(dx)$. The features therefore represent log densities up to a normalization constant, rather than densities themselves.

We learn a family of policies $\pi_\omega(\cdot \mid s,z)$, with actor parameters ω. Task vectors are drawn from a distribution p_Z that mixes random vectors with indices inferred from sampled goal-reaching rewards using the encoder in Appendix B.2.

For a minibatch $\{(s_i,a_i,s'_i,\text{terminal}_i)\}_{i=1}^n$, we use $\{s'_j\}_{j=1}^n$ as candidate future states and define

$$p_j(s,a,z) := \frac{\exp f_\theta(s,a,z,s'_j)}{\sum_{\ell=1}^n \exp f_\theta(s,a,z,s'_\ell)}. \tag{14}$$

The Bellman equation in (2) combines the observed next state with the distribution of later states. Its minibatch target and loss are

$$\begin{aligned} p_{ij} &:= p_j(s_i,a_i,z_i), \qquad \bar{p}'_{ij} := \bar{p}_j(s'_i,a'_i,z_i), \qquad a'_i \sim \pi_\omega(\cdot \mid s'_i,z_i), \\ t_{ij} &:= \text{sg}\left[(1-d_i)\mathbf{1}[j=i] + d_i\bar{p}'_{ij}\right], \qquad d_i := \gamma(1-\text{terminal}_i), \\ \mathcal{L}_{\text{DFB}} &:= -\frac{1}{n}\sum_{i,j} t_{ij}\log p_{ij}, \end{aligned} \tag{15}$$

where $\bar{p}_j$ uses target networks $(\bar{F},\bar{B})$ and sg stops gradients through the target. For non-terminal transitions, the weights are $1-\gamma$ and γ, as in the normalized SM definition. For terminal transitions, the target puts all mass on s'_i, treating that state as absorbing.

This update uses the next-state and bootstrapped-future structure of TD InfoNCE (?), with the policy conditioned on a task vector z. The following statement concerns exact updates; the minibatch loss approximates those updates.

Proposition 1 (Correct Bellman fixed point). Fix a task z and its policy π_z. Suppose normalization and expectations are computed exactly, every Bellman target has a density relative to ρ and is representable, and the population cross-entropy is minimized at every state-action pair. Assume the losses are finite. Then each update recovers the SM Bellman target, and repeated updates converge to M^{π_z}.

The proposition concerns a fixed policy, exact expectations over the reference distribution, and a target that the model class can represent. Two limits of that class matter in practice: finite logits assign positive mass to every candidate state, so successor measures with exact zeros, as in the process of fig. 2, are reached only in the limit; and under deterministic continuous dynamics the successor measure need not have a density with respect to the dataset marginal, so absolute continuity is an assumption. Neither concerns the proof; both bound where it applies.

The proof is given in Appendix A.1.

5.2 Using the learned distribution for control

For a state reward r, we compute its expectation under the predicted distribution, with the scaling from (5):

$$\widehat{Q}_{\theta,r}(s,a,z) := \frac{1}{1-\gamma}\sum_j p_j(s,a,z)r(s_j^+). \tag{16}$$

Applying the same calculation to the successor target gives a Bellman value target. Let $\bar{Q}'_{i,r} = (1-\gamma)^{-1}\sum_j \bar{p}'_{ij}r(s_j^+)$ denote the value obtained from the target-network probabilities. Substituting t_{ij} from (15) gives the target as

$$Y_{i,r} := \frac{1}{1-\gamma}\sum_j t_{ij}r(s_j^+) = \frac{1-d_i}{1-\gamma}r(s'_i) + d_i\bar{Q}'_{i,r}. \tag{17}$$

For a non-terminal transition, $d_i = \gamma$, so the target becomes the usual samples Bellman value target:

$$Y_{i,r} = r(s'_i) + \gamma \bar{Q}'_{i,r}, \tag{18}$$

For a terminal transition, the target gives $r(s'_i)/(1-\gamma)$, consistent with treating s'_i as absorbing. We therefore obtain the actor's value estimate directly from the learned probabilities.

During pretraining, we use $r_z(s^+) = B_\theta(s^+)^\top z$ and improve the policy by maximizing

$$\mathcal{J}_{\text{value}}(\omega) := \mathbb{E}_{\substack{s\sim\mathcal{D},\ z\sim p_Z \\ a\sim\pi_\omega(\cdot|s,z)}} \left[\widehat{Q}_{\theta,r_z}(s,a,z)\right]. \tag{19}$$

For offline training, we combine this value objective with behavior regularization. The full actor objectives and value scaling are given in Appendix C.2.

Task inference and deployment. The normalized parameterization breaks the FB identity that maps a downstream reward directly to a task vector, so we infer the task vector from reward-labelled offline states by a ridge regression of the reward on the centered backward features (Appendix B.2): with $\hat{b}$ the feature mean and c_r the ridge coefficients, the regression fits $\tilde{r}(s) = \bar{r} + (B_\theta(s) - \hat{b})^\top c_r$, and the task vector is the direction of c_r. The same regression produces the task vector of every goal-reaching task during pretraining. Two requirements are therefore separate. Equation (16) evaluates any reward under the represented policy, exactly for an exact successor model (Appendix B.1). The deployed actor, however, was trained for latent rewards $B_\theta(s^+)^\top z$ and is executed on the inferred task vector, so its action rankings are those of the fitted reward $\tilde{r}$; when a reward is poorly expressed by the backward features, the policy can be wrong even though the evaluation is right. At test time all networks remain frozen: we infer the task vector from the new reward and execute the frozen actor $\pi_\omega(\cdot \mid s, z_r)$ without additional training, environment interaction, or value computation, as in Algorithm 1; the value readout (16) is used only for diagnostics.

6 Experiments

Our experiments test whether FB's squared successor approximation can discard rare but control-critical action distinctions, and whether the Density-FB preserves those distinctions through normalization and the subsequent policy extraction process introduced in section 5. In section 6.1, we vary the rarity of the bottleneck and the representation capacity for FB and Density-FB for a set of didactic bottleneck MDPs with known action gaps and closed form SM solutions to test whether FB loses the correct action-order, and which of Density-FB's design decisions help it preserve this ordering. We then examine trained FB and Density-FB policies in section 6.2 to isolate the failure of FB in pick-and-place problems, testing whether errors in the learned critic's predictions coincide with these failures. Finally, we evaluate whether these improvements demonstrated by Density-FB translate to downstream manipulation performance across OGBench (?), and offline datasets collected in MetaWorld (?), and ManiSkill (?) (section 6.3). We additionally compare Density-FB against zero-shot baselines on reward inference tasks in the Deepmind Control suite (DMC) using ExoRL offline datasets.

Experimental protocol. All methods use states, are trained for 10^6 gradient steps on the same fixed offline dataset. We report results over 5 independent training seeds. For offline policy learning, we use the actor from ?, based on Flow Q-Learning (?). Success rates are averaged over the final three evaluations, with 10 episodes per task. Full implementation details are given in appendix C of the appendix.

Baselines. We compare against two groups of methods. 1. We consider methods that learn reusable representations from the offline dataset: FB (??), our primary baseline, together with HILP (?) and RLDP (?). These comparisons test whether the manipulation failure is specific to FB's successor approximation or is shared by alternative geometric and predictive representations. They are all trained using a task-independent representation through their backward embedding, and during inference, we specify the downstream goal reward on 10^4

offline transitions and use these reward-labeled transitions only to infer the corresponding task vector and extract a policy. 2. We compare against goal-conditioned offline RL methods: GCIQL, CRL, and QRL. These methods train directly for goal reaching using hindsight goals sampled from the offline data, and help us examine how well the zero-shot policies compare with methods trained explicitly for goal-reaching control.

6.1 Distinguishing actions at a control bottleneck

We use the eleven-state process in fig. 2 to examine whether learned successor representations preserve the difference between actions at the bottleneck. The continuation policy is fixed, and reward is 1 in cluster A (states $3, 4$) and 0 elsewhere. We first compare action values, then examine the failure using lemma 1, and finally ablate the changes introduced by Density-FB.

Do the critics distinguish the two actions? The true action values at state 2 are 1.67 for a_+ and 0 for a_-. FB assigns both actions a value of 0.54, whereas Density-FB predicts 1.67 and 0.01, respectively (fig. 3, right). Thus, FB treats the two actions as equally valuable, while Density-FB preserves the preference for entering the rewarded cluster.

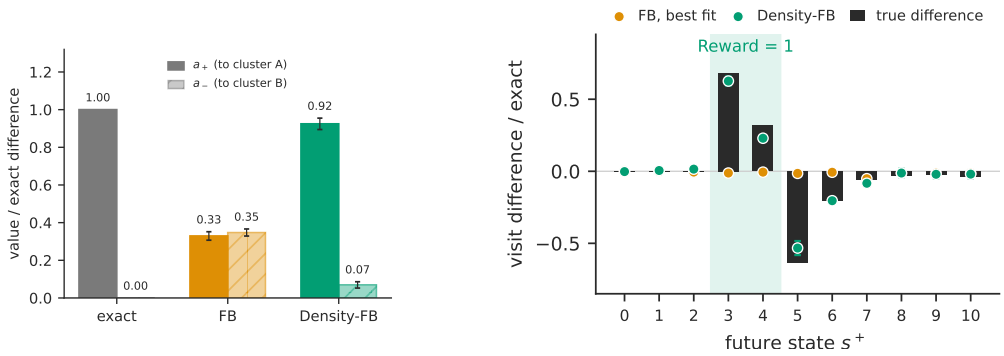


Figure 3: With three features, FB loses the action-value difference while Density-FB preserves it. Left: Predicted and exact action values at the bottleneck, showing means and 95% intervals over ten runs. Right: Differences $(a_+ - a_-)$ in discounted future visits. Black bars show the true difference; orange circles show the difference between least-squares fits to the exact successor densities, with the learned FB features held fixed. Even this best fit predicts no difference, missing the extra visits to rewarded states (shaded) that account for the true value difference of 1.67.

Does refitting recover the missing difference? We freeze the backward features of the trained rank-3 FB models and refit the two forward vectors to the exact successor densities by least squares, weighted by ρ. Across all ten runs, the fitted difference is zero at every future state (fig. 3, left). This illustrates lemma 1: the projection onto these features removes the entire difference between the two futures, including the extra visits to rewarded states 3 and 4. The best least-squares fit with these features therefore predicts a value difference of 0 instead of 1.67.

Which change in Density-FB helps preserve the difference? We vary the feature dimension and compare the components of Density-FB in fig. 4 and report $\widehat{G}_r/G_r$, the predicted action-value difference divided by the true difference. A value of 1 indicates exact recovery. Normalizing FB's target has little effect at the smaller dimensions. Applying a softmax to the bilinear scores improves recovery under both squared error and cross-entropy. With four features, both softmax variants recover essentially the full difference, while FB retains approximately 63%. FB improves as the feature dimension increases. These ablations identify the softmax parameterization as the main contributor to preserving the action-value difference with fewer features in this process.

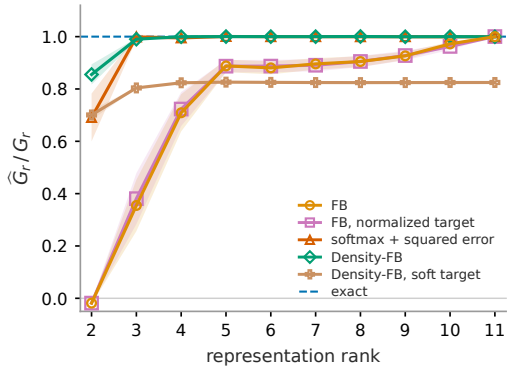


Figure 4: Softmax preserves the action-value difference with fewer features. We vary the feature dimension and measure the predicted value difference divided by its true value. The dashed line indicates recovery of this difference. Softmax improves recovery under both squared error and cross-entropy, while normalizing FB's target alone gives little improvement at smaller dimensions.

6.2 Does the phenomenon extend to manipulation?

We next examine where FB fails in manipulation tasks and whether the critic captures how changing an action changes the cube's future in the cube-single from OGBench. Following ?, we classify each evaluation episode as successful or by its first failed stage: reaching the cube (approach), grasping it (acquisition), keeping it held (maintain), or carrying it to the goal (transport). Most FB failures occur after grasping, while holding or carrying the cube. Density-FB has fewer failures at both stages (fig. 5a). We next examine the critic at states restored from the approach, holding, and carrying phases. For each state, we estimate two actions' future cube-position distributions through rollouts, taking each action and then following the policy represented by the critic. We compare the predicted difference between these distributions with the rollout estimate using cosine similarity. Both methods have alignment near zero during approach, but Density-FB has higher alignment during holding and carrying, with the largest separation during transport (fig. 5b).

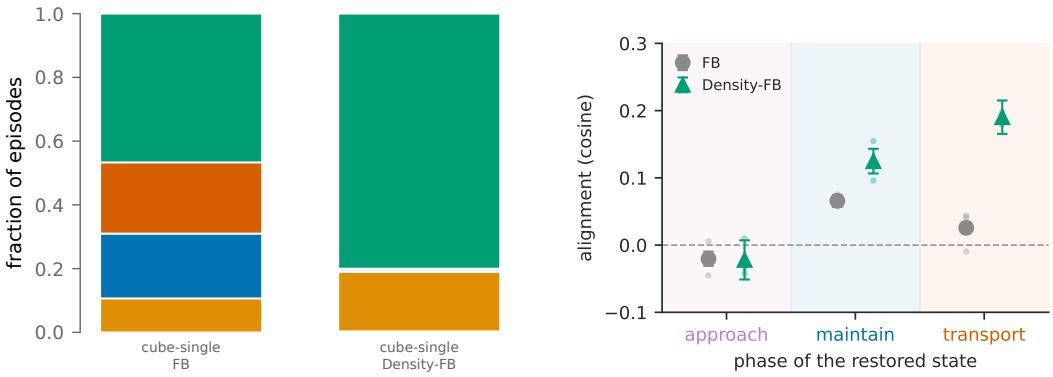


Figure 5: Failures and critic predictions during cube manipulation. (a) Episodes classified by their first failed stage: approach (reach the cube), acquisition (grasp it), maintain (keep it held), or transport (carry it to the goal). Completed tasks are shown as success. (b) Cosine similarity between the predicted and rollout-estimated differences in discounted future cube visits for two actions. Zero indicates no directional alignment. Large markers and error bars show means and standard errors over evaluated states; small markers show the three seed means.

6.3 Manipulation control across benchmarks

[Aditya: Update Goal-conditioned Density-FB numbers] We next evaluate Density-FB on 11 manipulation tasks from OGBench, MetaWorld, and ManiSkill, and locomotion tasks in DMC (table 1). Dataset details are provided in appendix F. Density-FB achieves higher

mean success than FB on every task, with average gains of 35.5, 43.6, and 29.7 percentage points on OGBench, MetaWorld, and ManiSkill, respectively. It also exceeds HILP and RLDP on all 11 tasks.

Against the goal-conditioned baselines, Density-FB achieves the highest mean success on seven tasks, including all five ManiSkill tasks. GCIQL performs better on cube-double, while QRL has higher mean success on the three MetaWorld tasks. For Density-FB, each downstream task is specified through rewards on 10^4 offline states, which are used to infer a task vector without further training.

Table 1: Performance across benchmarks. OGBench/MetaWorld/ManiSkill report success rate (%), while ExoRL (DMC runs) reports episode return (dense rewards). Goal-conditioned baselines do not apply to ExoRL and are shown as "–". All comparisons use the same offline datasets, architectures, actors, training budgets, and evaluation procedures. Results are mean ± standard deviation over five seeds. On the mean rows the spread is the standard deviation of the mean propagated from the cells above.

Task/Domain	FB	HILP	RLDP	GCIQL	CRL	QRL	Density-FB
OGBench (success rate, %)							
cube-single	45.6 ±5.9	71.1 ±5.7	27.6 ±3.7	68.0 ±6.0	19.0 ±2.0	5.0 ±1.0	81.3 ±3.8
cube-double	2.2 ±2.7	11.1 ±3.3	2.7 ±1.3	40.0 ±5.0	10.0 ±2.0	1.0 ±0.0	30.4 ±2.6
scene	10.2 ±4.1	39.3 ±5.7	10.2 ±3.2	51.0 ±4.0	19.0 ±2.0	5.0 ±1.0	52.7 ±3.3
OGBench mean	19.3 ±2.6	40.5 ±2.9	13.5 ±1.7	53.0 ±2.9	16.0 ±1.2	3.7 ±0.5	54.8 ±1.9
MetaWorld (success rate, %)							
push	30.0 ±8.6	34.7 ±4.1	60.0 ±13.0	4.0 ±3.3	18.4 ±5.9	80.8 ±2.4	80.7 ±6.6
bin-picking	13.3 ±18.9	10.0 ±12.8	19.6 ±34.4	0.0 ±0.0	0.0 ±0.0	98.0 ±2.2	88.0 ±6.5
box-close	66.7 ±14.3	42.0 ±8.2	47.3 ±12.7	26.0 ±21.8	32.4 ±17.5	84.4 ±4.1	72.0 ±17.3
MetaWorld mean	36.7 ±8.4	28.9 ±5.2	42.3 ±13.0	10.0 ±7.3	16.9 ±6.2	87.7 ±1.7	80.2 ±6.5
ManiSkill (success rate, %)							
push-cube	58.7 ±16.0	50.7 ±12.7	22.0 ±1.6	68.4 ±4.3	29.6 ±15.5	96.4 ±0.8	97.3 ±3.8
pick-cube	4.0 ±1.6	6.0 ±1.6	46.0 ±7.5	2.8 ±1.0	4.4 ±1.5	7.2 ±3.0	49.3 ±3.4
stack-cube	64.0 ±5.7	66.0 ±4.3	32.7 ±3.8	28.0 ±7.7	2.4 ±2.3	67.6 ±3.4	71.3 ±2.5
poke-cube	88.0 ±3.3	42.0 ±4.9	64.7 ±6.8	73.6 ±3.9	10.8 ±2.7	92.0 ±5.1	92.7 ±6.2
lift-peg	34.0 ±7.5	37.3 ±26.4	66.0 ±5.7	9.6 ±5.9	20.0 ±11.9	66.0 ±7.5	86.7 ±4.7
ManiSkill mean	49.7 ±3.8	40.4 ±6.0	46.3 ±2.5	36.5 ±2.3	13.4 ±4.0	65.8 ±2.0	79.5 ±1.9
mean over 11 manipulation tasks	37.9 ±2.9	37.3 ±3.2	36.3 ±3.7	33.8 ±2.4	15.1 ±2.5	54.9 ±1.0	72.9 ±2.1
ExoRL (DMC runs; episode return)							
walker	811.5 ±5.9	796.4 ±7.7	723.9 ±18.3	–	–	–	750.2 ±29.8
quadruped	595.6 ±9.1	694.8 ±11.0	665.0 ±13.9	–	–	–	655.9 ±15.9
point-mass	513.0 ±20.0	371.0 ±37.1	476.3 ±39.4	–	–	–	751.5 ±74.1
cheetah	672.7 ±4.9	618.3 ±5.8	575.6 ±44.9	–	–	–	539.9 ±42.0
mean over domains	648.2 ±4.1	620.1 ±8.4	610.2 ±13.5	–	–	–	674.4 ±22.9

Beyond manipulation. We also evaluate Density-FB on the ExoRL RND datasets (table 1). Performance is mixed: Density-FB exceeds the reported FB returns on quadruped and point-mass, but performs worse on walker and cheetah. It achieves the highest mean return on point-mass and the highest mean over the four domains, while FB leads on walker and cheetah and HILP on quadruped; on cheetah, whose rewards are speed thresholds, Density-FB trails all three reported methods.

7 Conclusion

[Aditya: Update] We introduced Density-FB, which applies a softmax to bilinear scores, learns the resulting successor distributions through a categorical Bellman objective, and evaluates actions by integrating rewards under these distributions. In controlled bottleneck problems, this parameterization preserves action-value differences with fewer features than FB. Across OGBench, MetaWorld, and ManiSkill, Density-FB achieves higher mean success than FB on all eleven manipulation tasks.

References

A Theoretical Analysis of Density-FB

A.1 Proof of the linear-approximation lemma

Proof. Use the notation of lemma 1, with $\langle f, g\rangle_\rho = \mathbb{E}_{s^+\sim\rho}[f(s^+)g(s^+)]$ and $\|f\|_\rho^2 = \langle f, f\rangle_\rho$. First, we establish that the least-squares fit is equivalent to an orthogonal projection onto the feature space. For either target density $m \in \{m_+, m_-\}$ and any feature coefficient vector $c \in \mathbb{R}^d$, we can decompose the squared approximation error as:

$$\|m - c^\top B\|_\rho^2 = \|m - \Pi_B m\|_\rho^2 + \|\Pi_B m - c^\top B\|_\rho^2.$$

This decomposition holds because the residual $(m - \Pi_B m)$ is, by definition of the projection Π_B, orthogonal to every linear combination of the backward features. The first term is independent of c, and the second is minimized when $c^\top B = \Pi_B m$. Thus, $\widehat{m}_\pm = \Pi_B m_\pm$. Next, we rely on the linearity of the projection operator to show that the difference of predicted futures is simply the projected difference of the true futures. Subtracting the two fitted densities yields:

$$\widehat{m}_+(s^+) - \widehat{m}_-(s^+) = \big[\Pi_B(m_+ - m_-)\big](s^+) = (\Pi_B \Delta m)(s^+).$$

Finally, the residual imapcts the Q-value estimates. Let ΔQ_r and $\Delta\widehat{Q}_r$ denote the true and fitted pairwise action-value differences (corresponding to G_r and $\widehat{G}_r$). By applying the identity in Equation (10) to both the true densities and our fitted approximations, the error in the estimated Q-value gap is simply the reward evaluated on the part of the future difference did not survive the projection:

$$\begin{aligned}\Delta Q_r - \Delta\widehat{Q}_r &= \frac{\langle (m_+ - m_-) - (\widehat{m}_+ - \widehat{m}_-), r\rangle_\rho}{1-\gamma} \\ &= \frac{\langle \Delta m - \Pi_B \Delta m, r\rangle_\rho}{1-\gamma}.\end{aligned}$$

□

B Reward Evaluation and Task Inference

B.1 Evaluating a reward under a represented policy

For a bounded state reward r and policy index z, define

$$Q_{\theta,r}(s,a,z) := \frac{1}{1-\gamma}\mathbb{E}_{s^+\sim\rho}[h_\theta(s,a,z,s^+)r(s^+)] = \frac{1}{1-\gamma}\frac{\mathbb{E}_{s^+\sim\rho}[\exp f_\theta(s,a,z,s^+)\, r(s^+)]}{\mathbb{E}_{s^+\sim\rho}[\exp f_\theta(s,a,z,s^+)]}. \tag{20}$$

If the predicted SM is exact, then

$$Q_{\theta,r}(s,a,z) = \frac{1}{1-\gamma}\int r(s^+)\, M^{\pi_z}(s,a,ds^+) = Q_r^{\pi_z}(s,a). \tag{21}$$

The estimate $\widehat{Q}_{\theta,r}$ in (16) approximates this model value.

B.2 Inferring a task index from reward samples

Given N_r states $s_j^+ \sim \rho$ and rewards $r_j = r(s_j^+)$, write $b_j = B_\theta(s_j^+)$ and $\bar{r} = N_r^{-1}\sum_j r_j$. Let $\widehat{b}$ and $\widehat{C}$ be the running feature mean and covariance maintained during pretraining. Set $v = \mathrm{tr}(\widehat{C})/d$ and $\lambda = 0.01v$. Form $\widetilde{C}$ by replacing eigenvalues of $\widehat{C}$ below $10^{-6}v$ by that threshold. The encoder solves

$$(\widetilde{C} + \lambda I)c_r = \frac{1}{N_r}\sum_{j=1}^{N_r}(b_j - \widehat{b})(r_j - \bar{r}), \qquad z_r = \sqrt{d}\,\frac{c_r}{\|c_r\|}. \tag{22}$$

If $\|c_r\| < 10^{-6}$, we return a fixed task vector of norm $\sqrt{d}$ instead.

The coefficients define the reward approximation

$$\widetilde{r}(s^+) = \bar{r} + (B_\theta(s^+) - \widehat{b})^\top c_r.$$

When the normalization in (22) is used, $B_\theta(s^+)^\top z_r$ is a positive affine transformation of $\widetilde{r}(s^+)$. It therefore preserves action rankings for the fitted reward. The encoder is used for sampled goal-reaching rewards during pretraining and for downstream rewards at evaluation. Random task vectors are sampled directly.

C Implementation Details

This appendix gives the implementation details omitted from section 5. We first describe the successor networks and the two-head implementation, then the policy extractor and zero-shot evaluation procedure. Environment-specific outcome coordinates and all hyperparameters are given last.

C.1 Successor networks

The forward and backward networks are MLPs. Each hidden layer uses Linear→GELU→LayerNorm, followed by a linear output layer.

The backward network B_θ has one output head and no output normalization or orthogonality regularizer. The forward network receives $[s, a, z]$ and has two heads,

$$F_\theta(s, a, z) \in \mathbb{R}^{2\times d}. \tag{23}$$

Each forward head defines its own normalized successor distribution using Equation (13). The representation loss in Equation (15) is computed independently for the two heads and then averaged.

Target networks $\bar{F}$ and $\bar{B}$ are updated after every gradient step using an exponential moving average with rate η. The bootstrap action is produced by the current actor and treated as a stop-gradient when constructing the successor target.

Combining the two heads. Each forward head $h \in \{1, 2\}$ defines its own normalized distribution $p^{(h)}(s, a, z)$ over the candidate next states, and each head is trained against its own target. The two heads are combined only after a reward has been integrated: for a reward r the per-head values are $q^{(h)}(s, a, z) = \sum_j p_j^{(h)}(s, a, z)\, r(s'_j)$ and the value used by the actor is

$$q(s, a, z) := \min\left\{q^{(1)}(s, a, z),\, q^{(2)}(s, a, z)\right\}. \tag{24}$$

Because each head remains a probability distribution, adding a constant to the reward adds the same constant to both heads' values and hence to q, so action rankings are invariant to reward shifts; taking the minimum of the two probability vectors instead would not have this property, since its total mass depends on the action. The normalization and fixed-point results of section 5 apply to each head.

C.2 Policy extraction

The learned successor distribution provides the action value used for policy improvement. For each forward head h, we compute

$$q^{(h)}(s, a, z) := \sum_{j=1}^{n} p_j^{(h)}(s, a, z) r_z(s'_j). \tag{25}$$

The value used by the actor is the minimum of the two heads' values, $q := \min_h q^{(h)}$, as in eq. (24). We normalize its scale using

$$s_q := \mathrm{sg}\left[\mathbb{E}\,|q - \mathbb{E}[q]| + \varepsilon\right], \qquad \varepsilon = 10^{-6}. \tag{26}$$

This normalization controls the scale of the value term relative to the behavior-regularization term.

Flow-based actor. For the main experiments, we use the flow-based policy extractor used in TD-JEPA (?), based on Flow Q-Learning (?).

A goal-agnostic velocity field $v_\psi(s, a, t)$ models the behavior policy. Given

$$x_0 \sim \mathcal{N}(0, I), \qquad x_1 = a, \qquad t \sim \mathcal{U}[0, 1], \tag{27}$$

we define

$$x_t := (1 - t)x_0 + tx_1, \tag{28}$$

and train the velocity field with

$$\mathcal{L}_{\text{flow}} := \mathbb{E}\left[\|v_\psi(s, x_t, t) - (x_1 - x_0)\|^2\right]. \tag{29}$$

Let

$$a_{\text{BC}} := \text{ODE}_\psi(s, \eta), \qquad \eta \sim \mathcal{N}(0, I), \tag{30}$$

denote the action obtained by integrating the learned velocity field for K Euler steps.

The one-step actor $\pi_\omega(s, z; \eta)$ is trained using

$$\mathcal{L}_\pi := -\frac{\mathbb{E}\left[q\left(s, \pi_\omega(s, z; \eta), z\right)\right]}{s_q} + \lambda_{\text{BC}}\mathbb{E}\left[\|\pi_\omega(s, z; \eta) - a_{\text{BC}}\|^2\right]. \tag{31}$$

The velocity field and the actor are optimized separately using Equations (29) and (31). Removing the value term from Equation (31) gives the corresponding behavior-cloning policy.

During the actor update, the successor networks are fixed. Gradients pass through the action input of F_θ to the actor, but the parameters of F_θ and B_θ are not updated.

DDPG+BC actor. For the alternative extractor, we use a Gaussian policy

$$\pi_\omega(\cdot \mid s, z) = \mathcal{N}\left(\mu_\omega(s, z), \sigma^2 I\right), \tag{32}$$

with fixed standard deviation σ.

Its objective is

$$\mathcal{L}_\pi = -\frac{\mathbb{E}\left[q(s, \mu_\omega(s, z), z)\right]}{s_q} - \alpha\mathbb{E}\left[\log \pi_\omega(a \mid s, z)\right]. \tag{33}$$

C.3 Reward inference and zero-shot evaluation

All zero-shot evaluations use frozen successor and policy networks. A downstream reward is introduced only after pretraining, and no gradient updates or additional environment interaction are performed.

For reward inference, we evaluate the downstream reward on 10^4 states sampled from the offline dataset and infer the corresponding task vector using the procedure described in section 5.2. [Aditya: State the exact regression used for z_r here so that this matches the main method exactly.]

To evaluate an action value, we sample a fixed pool

$$U_P = \{u_1, \ldots, u_P\} \tag{34}$$

from the dataset next-state marginal ρ and precompute $B_\theta(u_j)$. For a reward r, the finite-pool estimate is

$$\hat{q}_{\theta,r}(s, a, z) = \sum_{j=1}^{P} p_j(s, a, z)r(u_j). \tag{35}$$

The pool is sampled once using a fixed seed and reused throughout evaluation, with $P = 4096$; p_j is the combination of eq. (24) applied to the pool. These pool values are used

Table 2: Density-FB hyperparameters. Values are shared across environments unless stated otherwise.

	Symbol	Value
Successor model		
Latent dimension	d	50
Score temperature	τ	1.0
Tower hidden sizes	–	$(512, 512, 512)$
Hidden activation	–	GELU + LayerNorm
Forward heads	–	2
Backward output normalization	–	none
Orthogonality regularization	–	none
Immediate Bellman target	–	one-hot
Optimization		
Discount	γ	0.99
Batch size	n	256
Gradient steps	–	10^6
Optimizer	–	Adam
Learning rate	–	3×10^{-4}
Target EMA rate	η	0.005
Policy extraction		
Main extractor	–	flow-based actor
Actor hidden size	–	512
Velocity-field hidden size	–	512
Euler steps	K	10
Distillation weight	λ_{BC}	3.0
DDPG+BC weight	α	0.3
DDPG+BC policy std.	σ	0.2
Zero-shot evaluation		
Reward-inference states	–	10^4
Default value pool	P	4096
Pool sweep	P	$256, 1024, 4096, 16384$
Evaluation	–	3×10 episodes per task and seed

only for the diagnostics of section 6.2: at deployment the agent executes the frozen actor $\pi_\omega(\cdot \mid s, z_r)$ on the inferred task vector, as in Algorithm 1, and no value is computed. Direct goal conditioning is likewise a diagnostic: it uses the goal supplied by the environment to specify the task and therefore removes reward-to-task inference from the evaluation. All network parameters remain frozen in both evaluation modes.

C.4 Hyperparameters

C.5 Sampling task vectors for critic and actor training

Every training row of the critic and of the actor carries a task vector $z \in \mathbb{R}^d$ with $\|z\| = \sqrt{d}$; the forward network conditions on it, and for the actor it defines the reward the policy ascends. How z is drawn is a design decision, since the policy family is only trained on the task vectors it sees. We use FB's standard task-prior mix with one change in how the goal-derived half is produced. For each row, with probability $1 - p_{\mathrm{rand}}$ the task vector is the encoding of a hindsight goal: a future state of the row's own trajectory is drawn as the goal, the goal-reaching reward it defines (the hit kernel of the successor objective) is evaluated on the batch's candidate next states, and z is the ridge regression of that reward on the centered backward features of appendix B.2, computed with a slowly updated copy of the backward network and the running mean and covariance of its features. With probability p_{rand} the task vector is instead a fresh draw from the uniform distribution on the sphere of radius $\sqrt{d}$, so that the actor is trained across the whole conditioning space and not only on the slice reachable by hindsight goals. Both branches are treated as constants: no gradient flows into the backward network through z. The same encoding is used at test time on the

reward samples of the downstream task, so the goal-derived task vectors seen in training and the inferred task vectors seen at evaluation come from one map.

The actor's reward for a task vector is the centered latent reward $r_z(s^+) = (B_\xi(s^+) - \hat{\mu})^\top z/\sqrt{d}$, read with the same slow copy B_ξ and running mean $\hat{\mu}$ that produced z, so that the reward the actor ascends is the one the encoder fitted; reading it from the fast target network instead trains the actor against a drifting reward and lost most of the cube-single success in our ablations. For rows that kept a hindsight goal the exact goal kernel can be used in place of the latent reward; on cube-single this made no difference to success and we keep the latent reward throughout.

The fraction p_{rand} is the one knob of this mix. The manipulation results use $p_{\text{rand}} = 1/2$, FB's default. On ExoRL the tasks are dense velocity rewards whose task vectors lie far from every hindsight goal's, and drawing every training task vector from the sphere ($p_{\text{rand}} = 1$) raised walker and cheetah while lowering quadruped and point-mass; the per-domain choice is recorded in appendix C.6.

C.6 ExoRL settings per domain

The ExoRL cells of table 1 use, for every domain, the RND datasets, state observations, 10^6 gradient steps, five seeds, and task inference from 5×10^4 reward-labelled transitions, and the settings of table 2 except as listed here. Shared across domains: successor towers $(512, 512, 512)$, backward towers $(256, 256)$ with the output projected onto the sphere, target rate 0.005, learning rate 10^{-4}, actor learning rate 10^{-4}, discount 0.98, and a hit kernel over the full observation with the radius calibrated to the hit rate of the manipulation kernel (walker $0:24$ at 15.9, quadruped $0:78$ at 21.1). Per domain: point-mass uses discount 0.99, actor learning rate 10^{-6} and a kernel over the two position coordinates at radius 0.03, as every reference implementation does on this domain; walker uses task vectors drawn only from the sphere during training, the actor's value normalizer, a single successor tower of width 1024 and target rate 0.001, which raised its return from 724.6 to 750.2 and lowered quadruped and point-mass, so those two keep the shared settings; cheetah uses the walker settings with the hit kernel restricted to the nine velocity coordinates of the observation (columns 8–16, radius 7.26 at the same hit rate), which raised its return from 431 to 540, against 532 for a kernel over the forward-speed coordinate alone. The selection was made on the reported seeds.

D Bottleneck experiment details

Process and policies. All experiments use the eleven-state transition graph in fig. 2, with two deterministic actions per state, $\gamma = 0.8$, and a uniform reference distribution $\rho(s^+) = 1/11$. Reward is 1 on states $3, 4$ in cluster A and 0 elsewhere. The main example, P11, uses a continuation policy that chooses each action with probability $1/2$. For five additional settings, R1–R5, we keep the transitions fixed and independently draw the probability of the first action at each state from $\mathcal{U}(0.25, 0.75)$. The sampling procedure retains a draw only if the true action-value difference is at least 0.5 and the best rank-2 linear fit retains less than half of it. All five initial draws passed these conditions. Figure 6 shows the six policy settings.

Training and the fixed-feature check. We use exact population expectations and bootstrapped target networks, with ten training runs per setting and feature dimensions $d = 2, \ldots, 11$. For the check of lemma 1, we freeze the trained backward features and refit the two bottleneck forward vectors to the exact successor densities by ρ-weighted least squares. We evaluate the fitted action-value difference using (5).

Objectives of the ablation arms. Let M be the exact successor matrix (one row per state-action pair, mass $1/(1-\gamma)$), $\bar{M} = (1-\gamma)M$ its normalized rows, $\rho = 1/n$, and $X = nM$ the density ratios FB fits. All rows carry equal weight in every objective. FB fits $F_i B^\top$ to X_i by squared error; its population optimum is the rank-d truncated singular value decomposition of X, and the value is read as $F_i^\top z$ with $z = \rho\, B^\top r$. FB, normalized target

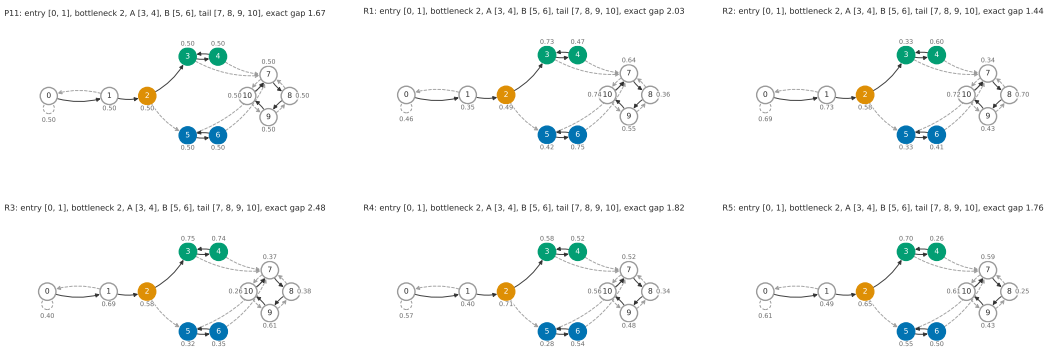


Figure 6: One transition graph, six continuation policies. P11 chooses each action with equal probability; R1–R5 use sampled probabilities. Numbers next to states give the probability of the first action (solid arrows); dashed arrows show the second. The bottleneck is orange, cluster A is green, and cluster B is blue. Reward is 1 in cluster A and 0 elsewhere.

fits $F_i B^\top$ to $\bar{M}_i$ with the same squared error and readout rescaled by $n/(1-\gamma)$, so its optimum coincides with FB's. Softmax + squared error predicts $p_i = \mathrm{softmax}(F_i B^\top)$ and minimizes $\frac{1}{2}\sum_j (p_{ij} - \bar{M}_{ij})^2$; Density-FB minimizes the cross-entropy $-\sum_j \bar{M}_{ij} \log p_{ij}$; both read the value as $\sum_j p_{ij} r_j/(1-\gamma)$. Density-FB, linear readout keeps Density-FB's trained F and B but reads $F_i^\top z$ as FB does. Density-FB, soft immediate target replaces the one-hot immediate term by a kernel over graph distance, $\exp(-\mathrm{dist}^2/(2 \cdot 0.5^2))$ row-normalized, so its fixed point is $\bar{M}$ smoothed by that kernel. TD-trained arms use the bootstrapped targets of these objectives, $(1-\gamma)P_i + \gamma\, P_i\, \bar{p}^{\pi}_{\text{target}}$ for the softmax arms and the corresponding squared TD error for the linear arms (with FB's orthogonality penalty at coefficient 1), a target network with rate 0.005, Adam, 5×10^4 steps, five seeds and two learning rates (10^{-3}, 10^{-2}), reported as the mean over the ten runs. The policy draws also change how long the two futures stay apart before they merge: on the three settings whose policies keep the process longer inside the clusters (R1, R3, R4), the difference between the futures carries 40–68% of its squared mass inside the top three singular directions of X, against none on the other three, which is why FB recovers part of the difference at rank 3 on those settings only.

Results across policies. Table 3 summarizes the feature dimension needed to reach 90% of the true action-value difference in each setting. Both softmax variants reach this threshold with two or three features, while FB requires five to ten.

Table 3: Features needed to retain the action-value difference. Smallest tested feature dimension for which the TD-trained model reaches $\widehat{G}_r/G_r \geq 0.9$. P11 uses the uniform policy; R1–R5 use sampled policies. A dash indicates that the threshold was not reached by $d = 11$.

Method	P11	R1	R2	R3	R4	R5
FB	5	9	5	5	10	10
FB, normalized target	5	10	5	5	9	10
Softmax + squared error	3	3	3	3	3	3
Density-FB	3	2	3	2	3	3
Density-FB, soft immediate target	–	–	–	–	–	–

E Reward extraction across successor representations

?? gives every method the task as a goal observation, which is the interface the goal-conditioned reference implementations are built for. This appendix gives the same frozen

checkpoints the task as a reward instead: each method draws 10^4 transitions from its own training data, labels them with the task reward, and solves for the conditioning vector whose implied values best explain those labels, through its own inference rule. Only successor representations appear here, because the comparison is about the reward-extraction step and the goal-conditioned baselines have none.

Table 4: Task given as a reward (success rate, %). The same 10^6-step checkpoints as ??; the conditioning vector is inferred from 10^4 reward-labelled dataset states, five tasks per environment, ten episodes per task, mean ± standard deviation over three seeds.

Task	FB	HILP	RLDP	Density-FB	Density-FB ^{oc}
cube-single	42.0 ±11.4	72.7 ±3.4	36.7 ±2.5	92.0 ±0.0	84.7 ±4.1
cube-double	4.7 ±0.9	8.0 ±0.0	2.0 ±1.6	14.7 ±4.1	34.0 ±8.6
scene	10.7 ±6.8	37.3 ±4.1	10.7 ±4.1	50.7 ±6.8	52.0 ±1.6
puzzle-3x3	4.0 ±3.3	0.7 ±0.9	9.3 ±7.5	5.3 ±2.5	6.7 ±1.9

^{oc}: the backward network reads the object-coordinate block of the observation instead of the full state. Every other column reads the full state. On cube-double reward extraction is capped near half of goal-conditioned success for every method: the environment re-permutes the cube-to-target assignment at every reset, while the reward samples encode one fixed assignment.

Two readings follow.

First, the reward-extraction step is where the successor representations separate. Density-FB leads on three of the four environments, and its margin over FB is larger here than under goal conditioning (92.0 against 42.0 on cube-single, 50.7 against 10.7 on scene).

Second, the two interfaces are not interchangeable, and which one a method prefers is itself informative. FB and RLDP lose most of their performance when the task arrives as a goal rather than a reward (42.0 → 17.3 and 36.7 → 10.7 on cube-single): evaluating their backward map at a single state is a harder query than averaging it against 10^4 reward-weighted samples. HILP moves the other way (72.7 → 86.0), since its features are trained to make state-to-state distances meaningful and a goal is its native query. Density-FB is stable across both (92.0 and 84.3), which is what a normalized successor measure predicts: the value is an average of the reward against a distribution, so it does not depend on whether the reward is concentrated at one state or spread over many.

F MetaWorld and ManiSkill datasets

This appendix describes how the offline datasets used in Section 6.3 were collected. The same environment wrapper is used for data collection and for evaluation, so the observation and action spaces, the control frequency, and the success criterion are identical in both.

Common protocol. Each dataset consists of N fixed-length trajectories of T environment steps, giving $N \times T \geq 10^6$ transitions per task (Table 5). Every trajectory pursues a single goal drawn once at reset from the environment's own goal distribution; the goal is never re-drawn within a trajectory. The behavior policy typically completes the task early and then holds its final configuration, so the remainder of the trajectory records the object at the goal. The fraction of transitions in which the object remains at the goal (the at-goal fraction) ranges from 0.76 to 0.95 across tasks; this is the persistent-occupancy structure discussed in Section 4. Each dataset is stored as one packed array file per task with fields observation, action, next_observation, reward, terminal (all zero; the horizon is fixed), timeout (set at $t = T - 1$), and success; MetaWorld additionally stores achieved_goal and goal_switch. Actions are clipped to $[-1, 1]$.

MetaWorld (push, bin-picking, box-close). We use the goal-observable V2 environments behind a wrapper replicating the collection setup of ?: action repeat 2 with summed rewards, a reset consisting of env.reset() followed by one zero-action step (which fills the previous-frame half of the 39-dimensional state), and 4-dimensional actions. The environment's internal path-length truncation is disabled so that every trajectory runs for the full $T = 500$

Table 5: Dataset sizes and behavior policies. N trajectories of T wrapper steps each; at-goal fraction = fraction of transitions with the object held at the goal; episode success = fraction of collected trajectories that reach the goal.

Suite	Task	N	T	At-goal fraction	Episode success
MetaWorld	push	2010	500	0.921	—
MetaWorld	bin-picking	2010	500	0.895	—
MetaWorld	box-close	2010	500	0.812	—
ManiSkill	push-cube	8000	125	0.949	0.997
ManiSkill	pick-cube	8000	125	0.906	0.990
ManiSkill	stack-cube	8000	125	0.756	0.940
ManiSkill	poke-cube	8000	125	0.858	0.948
ManiSkill	lift-peg	8000	125	0.783	0.921

wrapper steps (1000 simulator steps). Each task has $N = 2010$ trajectories (1,005,000 transitions), collected in 30 shards of 67 episodes with per-episode seeding of the global random number generator, from which MetaWorld draws its reset randomization.

The behavior policies are the scripted oracles shipped with MetaWorld. For push we use SawyerPushV2Policy, which follows the goal channel of the observation. For the two fixture tasks, the transport policies (SawyerPickPlaceV2Policy for bin-picking, SawyerBoxCloseV2Policy for box-close) are augmented with a deterministic grasp primitive that repairs the scripted grasp under action repeat 2: the gripper is aligned above the object to within 1.5–2 cm with hysteresis, descends open to the grasp depth (the mocap floor at $z \approx 0.065$ for the cube; 5 cm above the lid for the handle), closes, and hands control back to the scripted policy once the finger separation enters the measured carry band (0.35–0.60 for the cube, 0.25–0.40 for the lid). Per-episode Gaussian action noise is added with standard deviation $\sigma \sim \mathcal{U}[0, \sigma_{\max}]$, $\sigma_{\max} = 0.02$ for push and bin-picking and 0.01 for box-close.

The goal protocol is task-specific. push: one target per episode from the environment's own reset distribution (goal-space xy with z drawn from the empirical pool of reset heights). bin-picking: one placement into the goal bin, after which the object is held in the closed gripper. box-close: one placement of the lid onto the box, after which the lid is released and the hand hovers above it (holding the lid with the gripper closed makes the scripted controller oscillate). Fixtures are restored to their canonical pose at every reset and clamped to a wall-safe region (box $y \in [0.68, 0.78]$). Because the environment clips the goal dimensions of the observation (indices 36–38) to its goal-space box, which is degenerate for the fixture tasks, the true target is written into those dimensions on every stored frame. Success is $\|\text{obj} - \text{target}\| < r$ with $r = 0.06$, 0.06, and 0.08 m for the three tasks. Trajectories in which the object leaves the workspace (push: $|x| > 0.30$; bin-picking: $|x| > 0.45$; box-close: $|x| > 0.38$ or $y \notin [0.35, 0.90]$) are rejected at the first offending step and re-rolled, with at most 8 retries. The measured at-goal fractions are 0.921, 0.895, and 0.812, and no rejected trajectory was released into the datasets.

ManiSkill (push-cube, pick-cube, stack-cube, poke-cube, lift-peg). We use the ManiSkill3 (version 3.0.1) state-observation environments with the per-task control modes of ? (pd_ee_delta_pos for pick-cube, pd_ee_delta_pose otherwise), wrapped with CPUGymWrapper(ignore_terminations=True). The horizon is overridden to $T = 125$ steps, five times the 25-step evaluation horizon, so that the post-completion hold is recorded. Each task has $N = 8000$ trajectories (1,000,000 transitions), collected in 8 shards of 1000 episodes with env.reset(seed) seeding.

The behavior policies are the released multitask TD-MPC2 checkpoints of ? (model size B), with observations zero-padded to 128 dimensions and the 16-dimensional action head truncated to the task's action dimension. Actions are drawn from the stochastic policy head (eval_mode=False), with planning disabled (mpc=False) and no additional exploration noise or ϵ-mixing. The environment's own goal distribution is used unchanged. Tasks complete in roughly 10–20 steps and the policy then holds. The measured episode success

rates are 0.997, 0.990, 0.940, 0.948, and 0.921, and the at-goal fractions are 0.949, 0.906, 0.756, 0.858, and 0.783.

Goal coordinates in the observations, and terminals. The stored MetaWorld observations carry the task's target position in coordinates 36–38, and the ManiSkill observations carry the target position of their task (coordinates 25–27 for push-cube, 26–28 and 39–41 for pick-cube, and the corresponding coordinates for the other tasks). For every method in ?? these coordinates are zeroed in the observations and in every goal frame at training and evaluation time, so no agent receives the task through its input; the task reaches the zero-shot methods only through the reward-labelled transitions used for task inference, and the goal-conditioned baselines only through their goal input. All datasets are truncated by the time limit and contain no terminal transitions, so the terminal branch of Algorithm 1 is never exercised in our experiments.